sharun06 / Python-ML-skill-test-1-

Code Issues Pull requests Actions Projects Wiki Security Insights Settings

Python-ML-skill-test-1- / 240901103 skill test.ipynb

sharun06 Add files via upload 9206288 · now

1000 lines (1000 loc) · 272 KB

Preview Code Blame Raw

Open in Colab

1(A)

In [ ]:

```
scores = [78, 45, 90, 67, 52, 33, 81, 49, 60, 74, 88, 41, 55, 69, 72, 95, 38, 50, 64, 84]

average_score = sum(scores) / len(scores)
highest_score = max(scores)
lowest_score = min(scores)

print("Average Score:", average_score)
print("Highest Score:", highest_score)
print("Lowest Score:", lowest_score)

print("\nPass/Fail Status:")
for i, score in enumerate(scores, start=1):
    if score >= 50:
        print("Student", i, ":", score, "- Pass")
    else:
        print("Student", i, ":", score, "- Fail")

print("\nStudent Grades:")
for i, score in enumerate(scores, start=1):
    if score >= 85:
        grade = "A"
    elif score >= 70:
        grade = "B"
    elif score >= 60:
        grade = "C"
    elif score >= 50:
        grade = "D"
    else:
        grade = "F"

    print("Student", i, ":", score, "- Grade", grade)
```

```
Average Score: 64.25
Highest Score: 95
Lowest Score: 33

Pass/Fail Status:
Student 1 : 78 - Pass
Student 2 : 45 - Fail
Student 3 : 90 - Pass
Student 4 : 67 - Pass
Student 5 : 52 - Pass
Student 6 : 33 - Fail
Student 7 : 81 - Pass
Student 8 : 49 - Fail
Student 9 : 60 - Pass
Student 10 : 74 - Pass
Student 11 : 88 - Pass
Student 12 : 41 - Fail
Student 13 : 55 - Pass
Student 14 : 69 - Pass
Student 15 : 72 - Pass
Student 16 : 95 - Pass
Student 17 : 38 - Fail
Student 18 : 50 - Pass
Student 19 : 64 - Pass
Student 20 : 84 - Pass

Student Grades:
Student 1 : 78 - Grade B
Student 2 : 45 - Grade F
Student 3 : 90 - Grade A
Student 4 : 67 - Grade C
Student 5 : 52 - Grade D
Student 6 : 33 - Grade F
Student 7 : 81 - Grade B
Student 8 : 49 - Grade F
Student 9 : 60 - Grade C
Student 10 : 74 - Grade B
Student 11 : 88 - Grade A
Student 12 : 41 - Grade F
Student 13 : 55 - Grade D
Student 14 : 69 - Grade C
Student 15 : 72 - Grade B
Student 16 : 95 - Grade A
Student 17 : 38 - Grade F
Student 18 : 50 - Grade D
Student 19 : 64 - Grade C
Student 20 : 84 - Grade B
```

1(B)

In [ ]:

```
import numpy as np
A = np.array([[2, 3],
              [1, 2]])
```

```
b = np.array([5, 3])

x = np.linalg.solve(A, b)

print("Solution vector x:")
print(x)
```

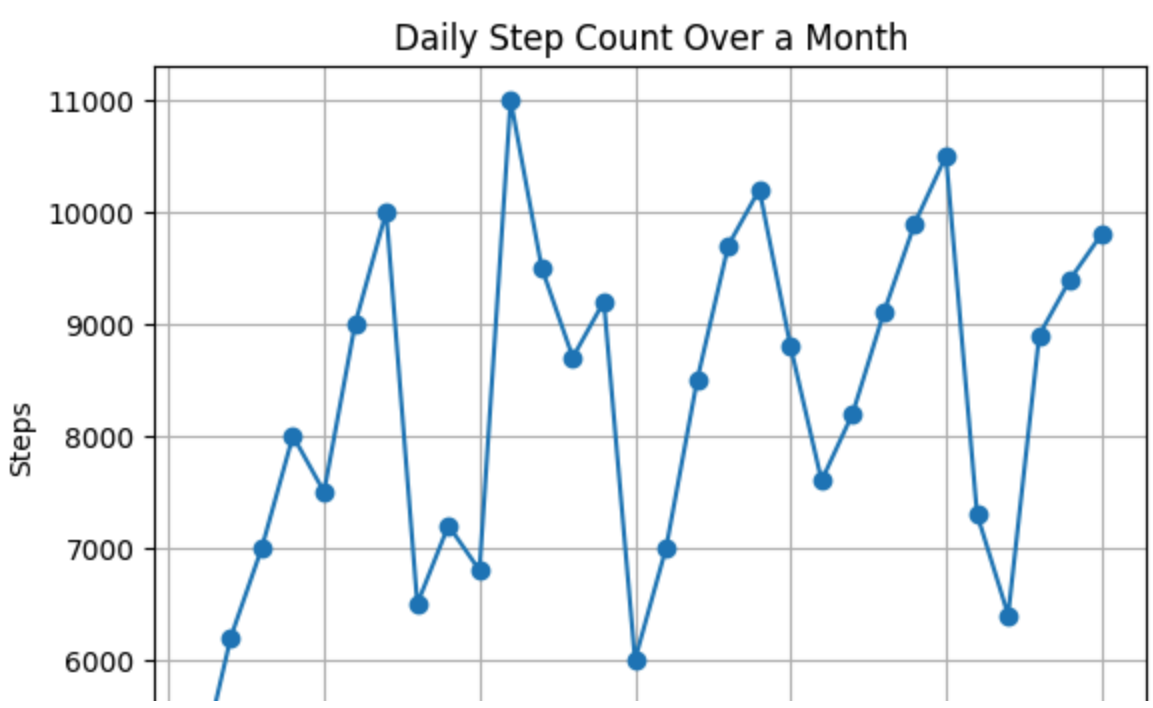
```
Solution vector x:
[1. 1.]
```

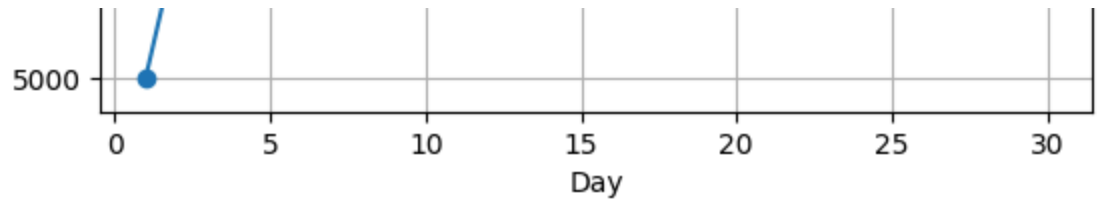
2(A)

In [ ]:

```
import numpy as np
import matplotlib.pyplot as plt

# Step count data for 30 days
steps = np.array([
5000, 6200, 7000, 8000, 7500, 9000, 10000,
6500, 7200, 6800, 11000, 9500, 8700, 9200,
6000, 7000, 8500, 9700, 10200, 8800, 7600,
8200, 9100, 9900, 10500, 7300, 6400, 8900,
9400, 9800
])

# Compute statistics
total_steps = np.sum(steps)
average_steps = np.mean(steps)
max_steps = np.max(steps)
min_steps = np.min(steps)

print("Total Steps:", total_steps)
print("Average Steps per Day:", average_steps)
print("Maximum Steps:", max_steps)
print("Minimum Steps:", min_steps)

# Identify high activity days
threshold = 9000
high_activity_days = np.where(steps > threshold)

print("\nHigh Activity Days (steps > 9000):")
print("Days:", high_activity_days[0] + 1)
print("Steps:", steps[high_activity_days])

# Analyze trend (increase or decrease)
trend = np.diff(steps)
print("\nDaily Step Changes:", trend)

# Visualization
days = np.arange(1, 31)

plt.plot(days, steps, marker='o')
plt.title("Daily Step Count Over a Month")
plt.xlabel("Day")
plt.ylabel("Steps")
plt.grid()
plt.show()
```

```
Total Steps: 248900
Average Steps per Day: 8296.666666666666
Maximum Steps: 11000
Minimum Steps: 5000

High Activity Days (steps > 9000):
Days: [ 7 11 12 14 18 19 23 24 25 29 30]
Steps: [10000 11000  9500  9200  9700 10200  9100  9900 10500  9400  9800]

Daily Step Changes: [ 1200   800  1000  -500  1500  1000 -3500   700  -400  4200 -1500  -800
   500 -3200  1000  1500  1200   500 -1400 -1200   600   900   800   600
 -3200  -900  2500   500   400]
```





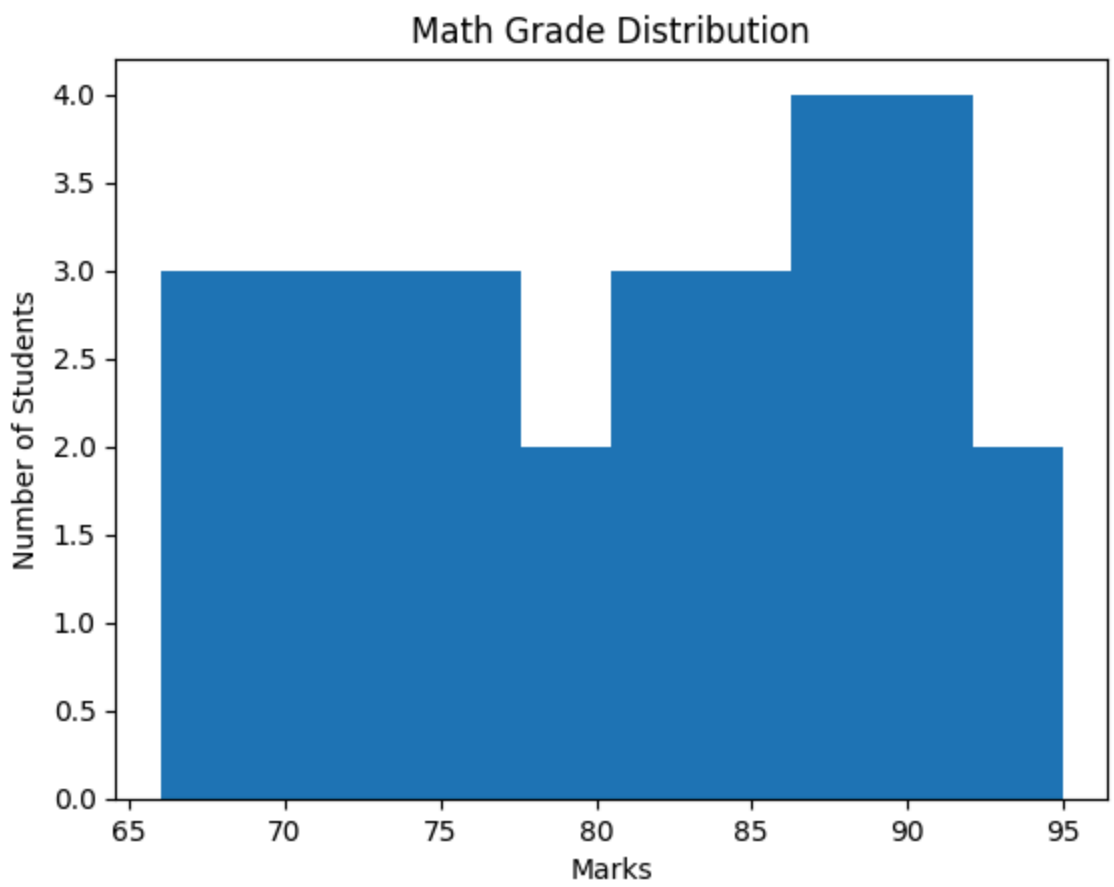
2(B)

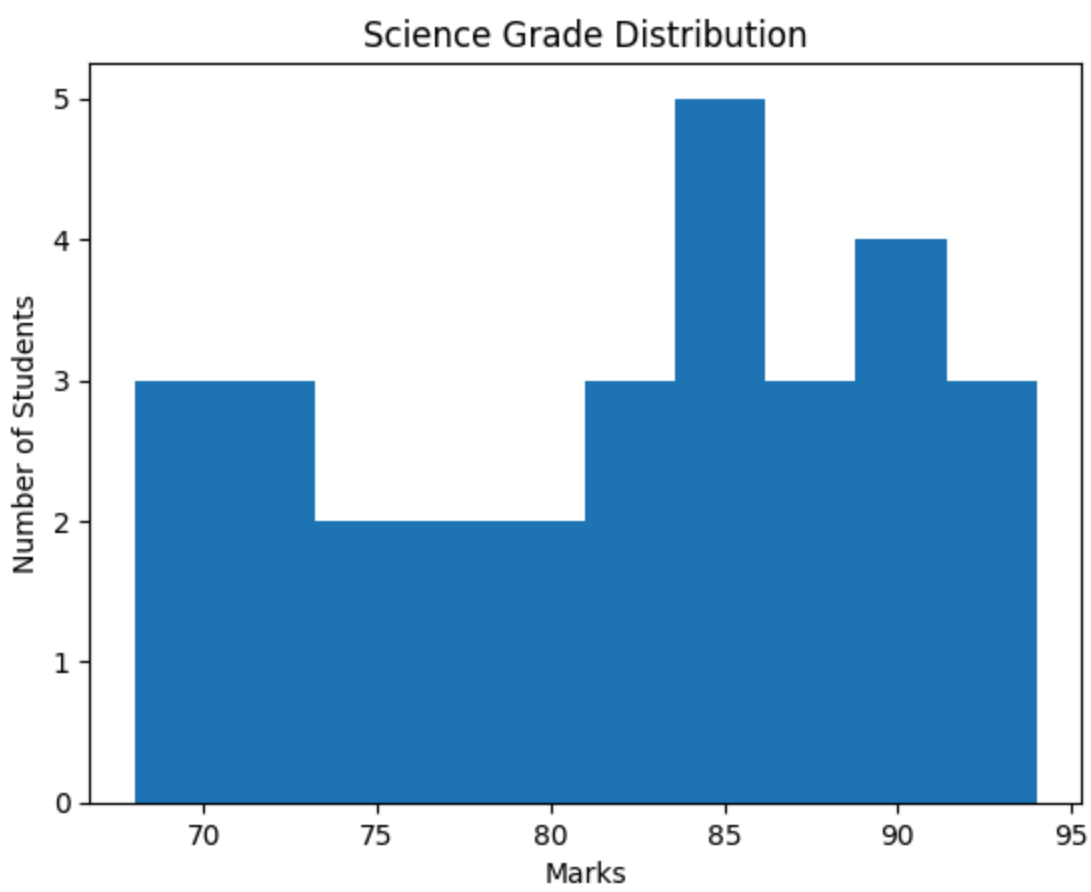
In [ ]:

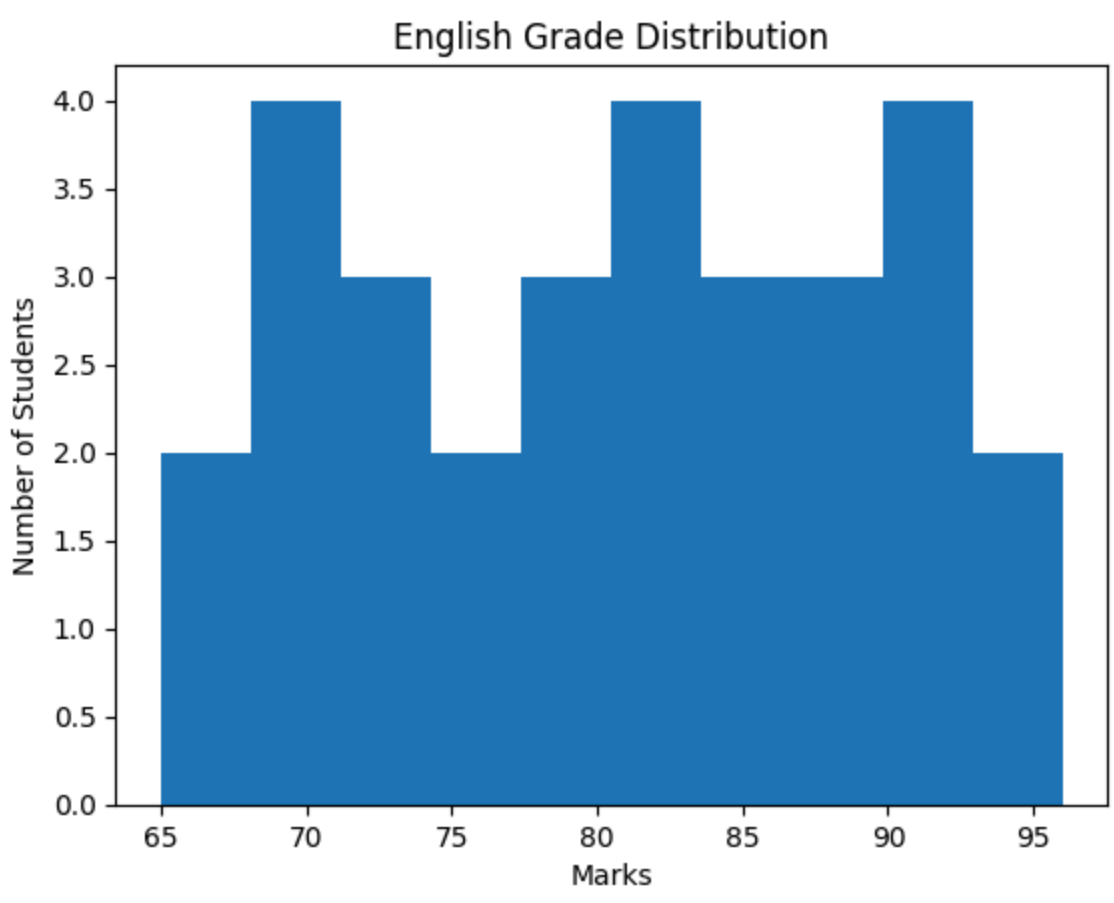
```
import numpy as np
import matplotlib.pyplot as plt

# Grades of 30 students (rows = students, columns = subjects)
# Columns: Math, Science, English
grades = np.array([
[78, 85, 82], [90, 88, 84], [67, 72, 70], [88, 90, 91], [76, 79, 75],
[92, 94, 89], [81, 83, 80], [73, 70, 72], [85, 87, 86], [69, 74, 71],
[95, 93, 96], [77, 80, 78], [84, 82, 83], [91, 89, 90], [66, 68, 65],
[72, 75, 74], [87, 85, 88], [79, 81, 80], [93, 92, 94], [70, 73, 69],
[82, 84, 83], [88, 90, 87], [76, 78, 77], [85, 86, 84], [90, 91, 92],
[68, 70, 67], [74, 76, 73], [89, 88, 90], [83, 85, 82], [71, 72, 70]
])

subjects = ["Math", "Science", "English"]

# Subject-wise statistics
average = np.mean(grades, axis=0)
highest = np.max(grades, axis=0)
lowest = np.min(grades, axis=0)

print("Subject-wise Statistics")
for i in range(3):
    print(subjects[i])
    print("Average:", average[i])
    print("Highest:", highest[i])
    print("Lowest:", lowest[i])
    print()

# Overall statistics
overall_avg = np.mean(grades)
overall_max = np.max(grades)
overall_min = np.min(grades)

print("Overall Average Grade:", overall_avg)
print("Overall Highest Grade:", overall_max)
print("Overall Lowest Grade:", overall_min)

# Total and average score per student
total_scores = np.sum(grades, axis=1)
average_scores = np.mean(grades, axis=1)

# Identify top-performing student
top_student = np.argmax(total_scores)

print("\nTop Performing Student (based on total marks):")
print("Student Number:", top_student + 1)
print("Total Score:", total_scores[top_student])

# Visualization using histograms
plt.hist(grades[:,0], bins=10)
plt.title("Math Grade Distribution")
plt.xlabel("Marks")
plt.ylabel("Number of Students")
plt.show()

plt.hist(grades[:,1], bins=10)
plt.title("Science Grade Distribution")
plt.xlabel("Marks")
plt.ylabel("Number of Students")
plt.show()

plt.hist(grades[:,2], bins=10)
plt.title("English Grade Distribution")
plt.xlabel("Marks")
plt.ylabel("Number of Students")
plt.show()
```

```
Subject-wise Statistics
Math
Average: 80.63333333333334
Highest: 95
Lowest: 66

Science
Average: 82.0
Highest: 94
Lowest: 68

English
```

```
Average: 80.73333333333333
Highest: 96
Lowest: 65

Overall Average Grade: 81.12222222222222
Overall Highest Grade: 96
Overall Lowest Grade: 65

Top Performing Student (based on total marks):
Student Number: 11
Total Score: 284
```







3(A)

In [ ]:

```
import numpy as np

# Enter impedance values (complex numbers)
# Example components: Resistor, Inductor, Capacitor
Z1 = complex(10, 0) # Resistor (10 Ω)
Z2 = complex(0, 5) # Inductor (j5 Ω)
Z3 = complex(0, -3) # Capacitor (-j3 Ω)

# Store impedances in a list
impedances = np.array([Z1, Z2, Z3])

print("Impedances:", impedances)

# ---- SERIES IMPEDANCE ----
Z_series = np.sum(impedances)

print("\nTotal Impedance in Series:", Z_series)

# ---- PARALLEL IMPEDANCE ----
Z_parallel = 1 / np.sum(1 / impedances)

print("Total Impedance in Parallel:", Z_parallel)

# ---- OPTIONAL USER INPUT ----
choice = input("\nEnter circuit type (series/parallel): ")

if choice == "series":
    print("Total Circuit Impedance:", Z_series)

elif choice == "parallel":
    print("Total Circuit Impedance:", Z_parallel)

else:
    print("Invalid choice")
```

```
Impedances: [10.+0.j  0.+5.j  0.-3.j]

Total Impedance in Series: (10+2j)
Total Impedance in Parallel: (3.6000000000000014-4.800000000000001j)

Enter circuit type (series/parallel): parallel
Total Circuit Impedance: (3.6000000000000014-4.800000000000001j)
```

3(B)

In [ ]:

```
import numpy as np

# Stock A price data (5 days × 3 features: Open, High, Low)
stock_A = np.array([
[100, 105, 98],
[102, 108, 101],
[104, 110, 103],
[103, 107, 100],
[105, 112, 104]
])

# Stock B price data (5 days × 3 features: Open, High, Low)
stock_B = np.array([
[98, 103, 95],
[101, 106, 99],
[102, 109, 100],
[100, 104, 97],
[103, 108, 101]
])

print("Stock A Data:\n", stock_A)
print("\nStock B Data:\n", stock_B)

# Element-wise sum
sum_matrix = stock_A + stock_B

print("\nElement-wise Sum of Stock Prices:\n", sum_matrix)

# Element-wise difference
difference_matrix = stock_A - stock_B

print("\nElement-wise Difference of Stock Prices:\n", difference_matrix)
```

```
Stock A Data:
 [[100 105  98]
 [102 108 101]
 [104 110 103]
 [103 107 100]
 [105 112 104]]

Stock B Data:
 [[ 98 103  95]
 [101 106  99]
```

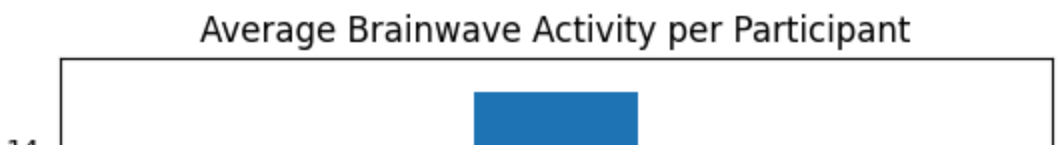
```
 [102 109 100]
 [100 104  97]
 [103 108 101]]

Element-wise Sum of Stock Prices:
 [[198 208 193]
 [203 214 200]
 [206 219 203]
 [203 211 197]
 [208 220 205]]

Element-wise Difference of Stock Prices:
 [[2 2 3]
 [1 2 2]
 [2 1 3]
 [3 3 3]
 [2 4 3]]
```

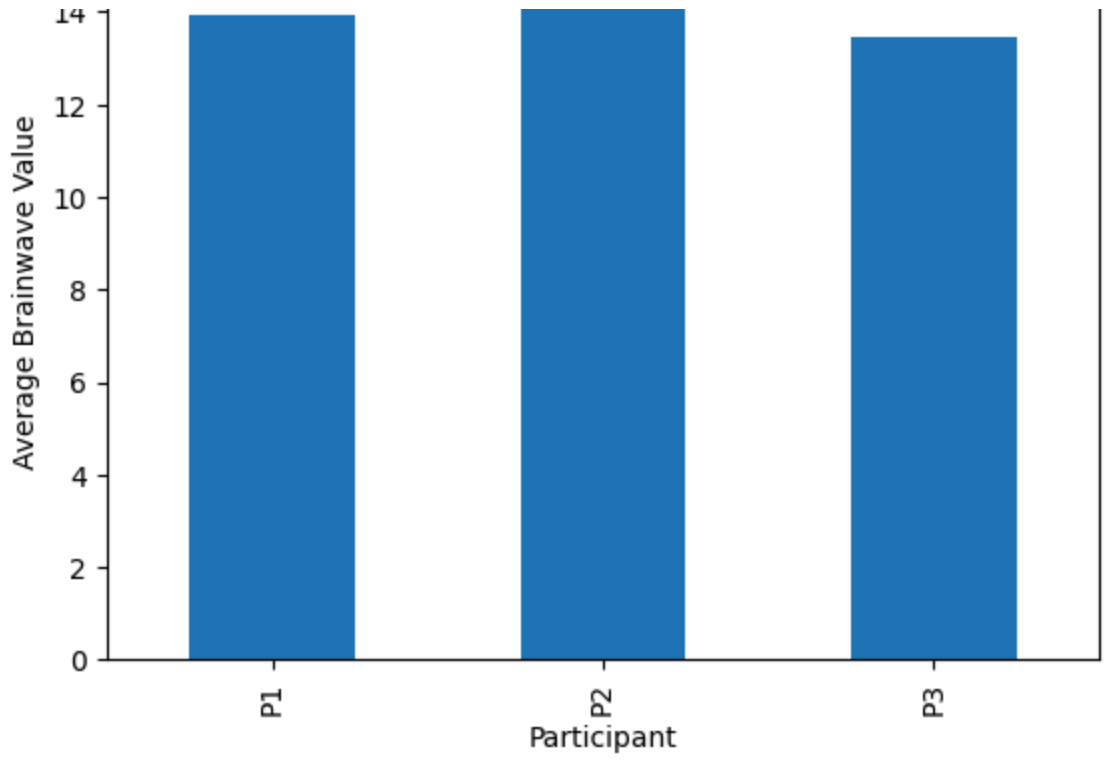
4(A)

In [ ]:

```python
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

# Example EEG dataset
data = {
    "Participant": ["P1","P1","P1","P2","P2","P2","P3","P3","P3"],
    "Electrode": ["E1","E2","E3","E1","E2","E3","E1","E2","E3"],
    "Brainwave": [12.5, 15.2, 14.1, 13.3, 16.5, 15.8, 11.9, 14.7, 13.8]
}

# Load data into Pandas DataFrame
df = pd.DataFrame(data)

print("EEG Data:\n")
print(df)

# Statistical analysis
stats = df.groupby(["Participant","Electrode"])["Brainwave"].agg(["mean","var","max"])

print("\nStatistical Analysis (Mean, Variance, Peak):\n")
print(stats)

# Identify overall trends
participant_mean = df.groupby("Participant")["Brainwave"].mean()

print("\nAverage Brainwave Activity per Participant:\n")
print(participant_mean)

# Visualization of trends
participant_mean.plot(kind="bar", title="Average Brainwave Activity per Participant")
plt.xlabel("Participant")
plt.ylabel("Average Brainwave Value")
plt.show()
```

```
EEG Data:

  Participant Electrode  Brainwave
0          P1        E1       12.5
1          P1        E2       15.2
2          P1        E3       14.1
3          P2        E1       13.3
4          P2        E2       16.5
5          P2        E3       15.8
6          P3        E1       11.9
7          P3        E2       14.7
8          P3        E3       13.8

Statistical Analysis (Mean, Variance, Peak):

                      mean  var   max
Participant Electrode
P1          E1        12.5  NaN  12.5
            E2        15.2  NaN  15.2
            E3        14.1  NaN  14.1
P2          E1        13.3  NaN  13.3
            E2        16.5  NaN  16.5
            E3        15.8  NaN  15.8
P3          E1        11.9  NaN  11.9
            E2        14.7  NaN  14.7
            E3        13.8  NaN  13.8

Average Brainwave Activity per Participant:

Participant
P1    13.933333
P2    15.200000
P3    13.466667
Name: Brainwave, dtype: float64
```





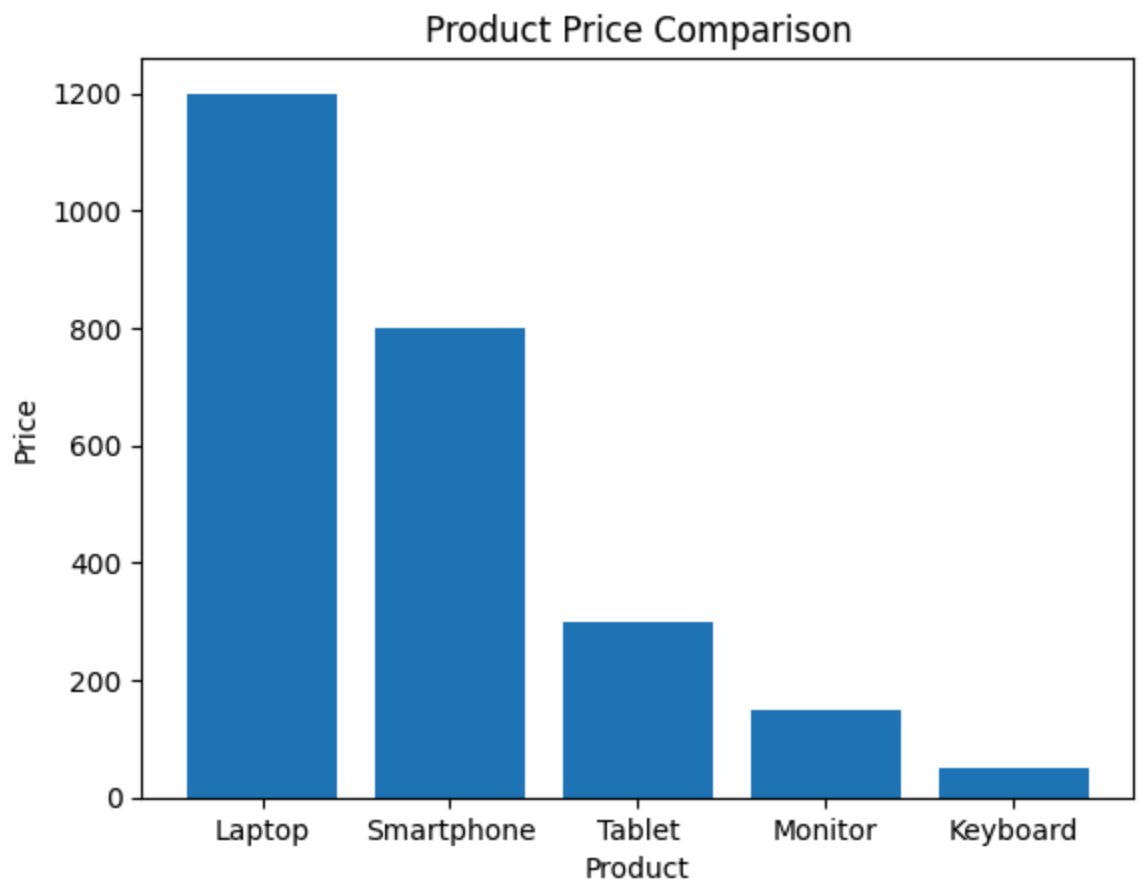
4(B)

In [ ]:

```
import pandas as pd
import matplotlib.pyplot as plt

# Given inventory data
data = {
'Product': ['Laptop', 'Smartphone', 'Tablet', 'Monitor', 'Keyboard'],
'Price': [1200, 800, 300, 150, 50],
'Quantity': [10, 20, 15, 25, 50]
}

# Create DataFrame
df = pd.DataFrame(data)

print("Inventory Data:\n")
print(df)

# Operation 1: Calculate total inventory value
df['Total_Value'] = df['Price'] * df['Quantity']
print("\nTotal Value of Each Product:\n")
print(df[['Product','Total_Value']])

# Operation 2: Average price
avg_price = df['Price'].mean()
print("\nAverage Price:", avg_price)

# Operation 3: Maximum price product
max_price_product = df.loc[df['Price'].idxmax()]
print("\nProduct with Maximum Price:\n", max_price_product)

# Operation 4: Minimum price product
min_price_product = df.loc[df['Price'].idxmin()]
print("\nProduct with Minimum Price:\n", min_price_product)

# Operation 5: Total quantity of products
total_quantity = df['Quantity'].sum()
print("\nTotal Quantity of All Products:", total_quantity)

# Plot: Price of products
plt.bar(df['Product'], df['Price'])
plt.title("Product Price Comparison")
plt.xlabel("Product")
plt.ylabel("Price")
plt.show()
```

```
Inventory Data:

      Product  Price  Quantity
0      Laptop   1200        10
1  Smartphone    800        20
2      Tablet    300        15
3     Monitor    150        25
4    Keyboard     50        50

Total Value of Each Product:

      Product  Total_Value
0      Laptop        12000
1  Smartphone        16000
2      Tablet         4500
3     Monitor         3750
4    Keyboard         2500

Average Price: 500.0

Product with Maximum Price:
```

```
 Product        Laptop
Price            1200
Quantity           10
Total_Value     12000
Name: 0, dtype: object

Product with Minimum Price:
 Product        Keyboard
Price                50
Quantity             50
Total_Value        2500
Name: 4, dtype: object

Total Quantity of All Products: 120
```

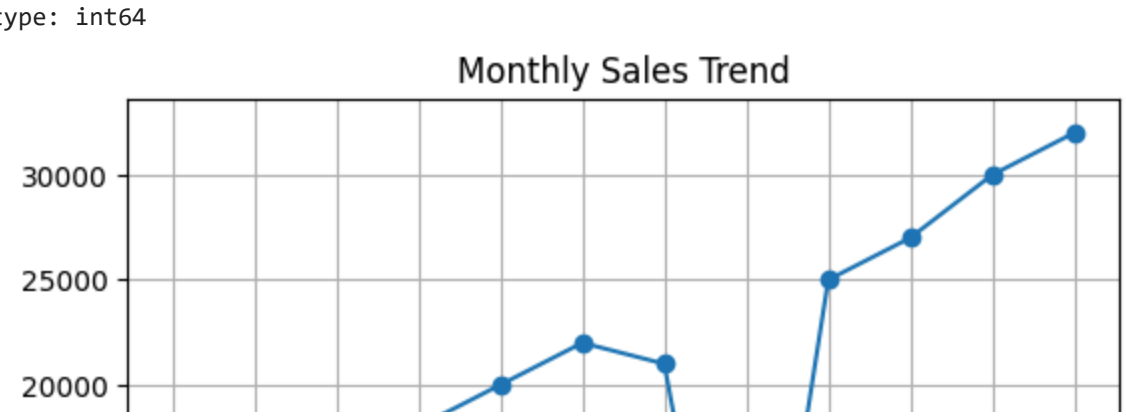


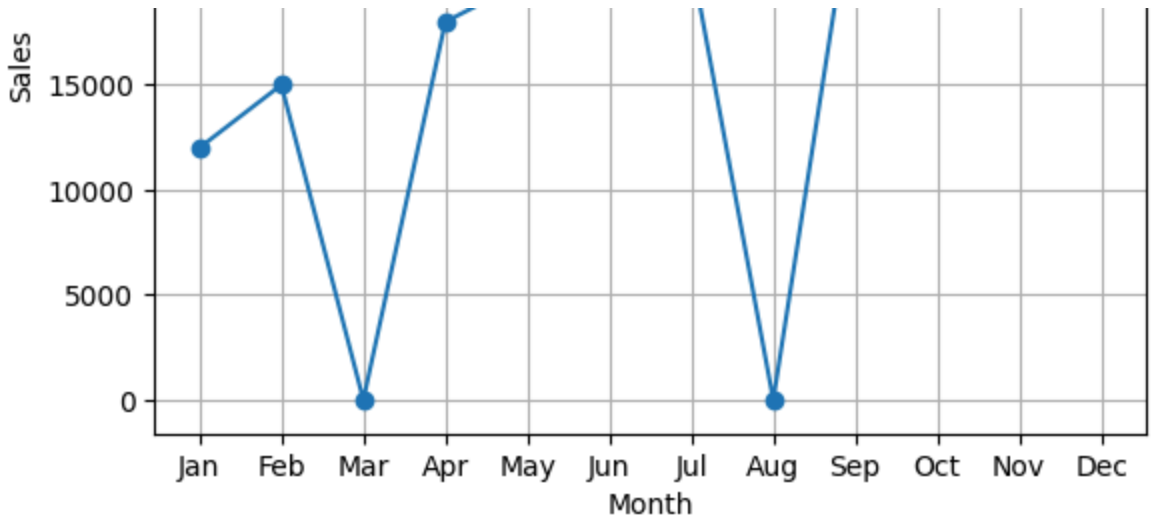
5(A)

In [ ]:

```python
import pandas as pd
import matplotlib.pyplot as plt

# 1. Read the CSV file
df = pd.read_csv("sales_data.csv")

# 2. Display first 5 rows
print("First 5 Rows of Dataset:")
print(df.head())

# 3. Check for missing values
print("\nMissing Values in Dataset:")
print(df.isnull().sum())

# Handle missing values by filling with 0
df.fillna(0, inplace=True)

# 4. Visualize monthly sales trend
plt.plot(df['Month'], df['Sales'], marker='o')
plt.title("Monthly Sales Trend")
plt.xlabel("Month")
plt.ylabel("Sales")
plt.grid(True)
plt.show()
```

```
First 5 Rows of Dataset:
  Month    Sales
0   Jan  12000.0
1   Feb  15000.0
2   Mar      NaN
3   Apr  18000.0
4   May  20000.0

Missing Values in Dataset:
Month    0
Sales    2
dtype: int64
```



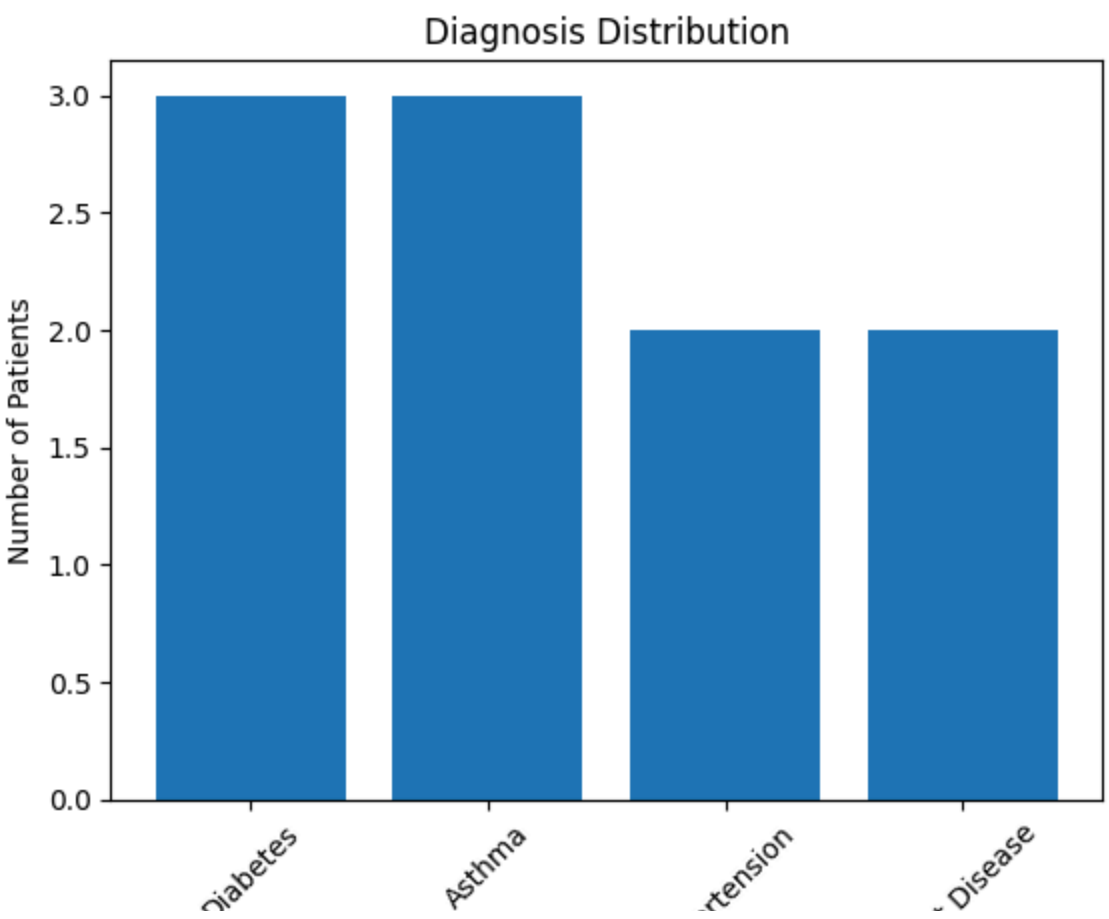


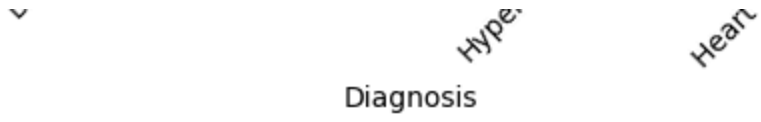
5(B)

In [ ]:

```
import pandas as pd
import matplotlib.pyplot as plt

# 1. Load the dataset
df = pd.read_csv("healthcare_data.csv")

# Display column names and data types
print("Column Names and Data Types:\n")
print(df.dtypes)

# 2. Calculate average age of patients
average_age = df["Age"].mean()
print("\nAverage Age of Patients:", average_age)

# 3. Analyze diagnosis distribution
diagnosis_counts = df["Diagnosis"].value_counts()

print("\nDiagnosis Distribution:\n")
print(diagnosis_counts)

# Visualization of diagnosis distribution
plt.bar(diagnosis_counts.index, diagnosis_counts.values)
plt.title("Diagnosis Distribution")
plt.xlabel("Diagnosis")
plt.ylabel("Number of Patients")
plt.xticks(rotation=45)
plt.show()
```

```
Column Names and Data Types:

Patient_ID     int64
Age            int64
Gender        object
Diagnosis     object
dtype: object

Average Age of Patients: 43.0

Diagnosis Distribution:

Diagnosis
Diabetes         3
Asthma           3
Hypertension     2
Heart Disease    2
Name: count, dtype: int64
```




Hype
Heart
Diagnosis